Diseño y Análisis de Algoritmos

Ayudantía 4: Hashing y Búsqueda de Patrones (KMP)

Ayud. Ferran Rojas Andreu

1. (A) División + encadenamiento

Valores hash.

k	22	12	14	5	20	7
$h(k)$	1	5	0	5	6	0

Tabla final. Insertando al inicio:

$T[0] = [7, 14], \quad T[1] = [22], \quad T[2] = \varnothing, \quad T[3] = \varnothing, \quad T[4] = \varnothing, \quad T[5] = [5, 12], \quad T[6] = [20].$

$\alpha = 6/7$.

Búsquedas. 12: $h = 5$, cadena $[5, 12]$, 2 comparaciones. 22: $h = 1$, $[22]$, 1 comparación. 7: $h = 0$, $[7, 14]$, 1 comparación. Buscar 13: $h = 6$, cadena $[20]$, se recorre completa y no se encuentra.

Eliminación. En $T[0] = [14, 7, 0]$: eliminar 14 (cabeza) deja $T[0] = [7, 0]$; eliminar 7 (medio) deja $T[0] = [0]$; eliminar 0 (cola) deja $T[0] = \varnothing$. Siempre se busca primero la clave y luego se actualizan los punteros.

Duplicados y casos límite. Si se permite duplicar 22, $T[1] = [22, 22]$; una búsqueda devuelve la primera copia y hay que decidir si la eliminación borra una o todas. Con $m = 1$ todas las claves caen en una única lista: la tabla se degenera en una lista enlazada y la búsqueda es $\Theta(n)$. Si $n > m$, el encadenamiento sigue funcionando, pero el largo medio de las cadenas es $\alpha > 1$ y el peor caso es $\Theta(n)$. Con listas ordenadas se puede detener antes una búsqueda fallida, pero la inserción deja de ser $O(1)$ y pasa a $O(L)$.

2. (B) División + direccionamiento abierto (sondeo lineal)

Sondeos.

k	22	12	14	5	20	7
sondeos	1	5	0	5, 6	6, 0, 1, 2	0, 1, 2, 3

Tabla final.

$$0 : 14, \quad 1 : 22, \quad 2 : 20, \quad 3 : 7, \quad 4 : \varnothing, \quad 5 : 12, \quad 6 : 5.$$

$\alpha = 6/7$. El sondeo lineal forma el bloque $0, 1, 2, 3$; cualquier clave que caiga en él recorre el bloque completo (agrupamiento primario).

Búsquedas.

- 20: $h = 6$; sondea $6, 0, 1, 2$ y lo encuentra. 4 sondeos.
- 13: $h = 6$; sondea $6, 0, 1, 2, 3, 4$ y termina en EMPTY. 6 sondeos; no está.

Eliminación. Eliminar 22 deja DELETED en 1. Para buscar 7 ($h = 0$), se sondea 0 (14), 1 (DELETED), 2 (20), 3 (7): 4 sondeos. Si la casilla 1 fuera EMPTY, la búsqueda se detendría allí y diría falsamente que 7 no está.

Reutilizar Deleted. Insertar 8: $h = 1$. Si no se permiten duplicados, primero se busca 8 (sondea $1, 2, 3, 4$ y termina en EMPTY); luego se inserta en 1. Si se permiten, se inserta directamente.

Tabla llena y rehash. Si se insertan $0, \ldots, 6$, la tabla queda llena; insertar 7 sondea las 7 casillas y falla. Una búsqueda fallida también debe detenerse tras m sondeos. Con α alto o muchos DELETED, se crea una tabla prima más grande, se reinsertan todas las claves recalculando $h(k)$ y se descartan las tumbas. Un duplicado de 22 se detecta al buscarlo y se aplica la política elegida.

3. (C) Hashing universal + encadenamiento

Valores.

k	22	12	14	5	20	7
$ak + b$	70	40	46	19	64	25
$(ak + b)$ mód 29	12	11	17	19	6	25
$h_{a,b}(k)$	5	4	3	5	6	4

Cadenas.

$T[0] = \varnothing, \quad T[1] = \varnothing, \quad T[2] = \varnothing, \quad T[3] = [14], \quad T[4] = [7, 12], \quad T[5] = [5, 22], \quad T[6] = [20].$

$\alpha = 6/7$. Buscar 22 en $[5, 22]$ cuesta 2 comparaciones; buscar 7 en $[7, 12]$ cuesta 1. Eliminar 5 deja $T[5] = [22]$.

Demostración. Para $x \neq y$, sean $r_x = (ax + b)$ mód 29 y $r_y = (ay + b)$ mód 29. Al variar (a, b), el par (r_x, r_y) es uniforme sobre los $29 \cdot 28$ pares de residuos distintos: dado $r_x \neq r_y$, se despeja $a \equiv (r_x - r_y)(x - y)^{-1}$ (mód 29) y $b \equiv r_x - ax$ (mód 29). Hay colisión si $r_x \equiv r_y$ (mód 7). Como $29 = 4 \cdot 7 + 1$, una clase módulo 7 tiene tamaño 5 y las otras seis tienen tamaño 4; los pares ordenados distintos en la misma clase son

$$5 \cdot 4 + 6 \cdot (4 \cdot 3) = 92.$$

Por lo tanto

$$\Pr[h_{a,b}(x) = h_{a,b}(y)] = \frac{92}{29 \cdot 28} = \frac{23}{203} < \frac{1}{7} = \frac{1}{m}.$$

La cota general es $\leq 1/m$.

Casos que rompen la familia. Si $a = 0$, $h_{0,b}(k) = b$ mód m es constante y todas las claves colisionan. Si p no es primo, $x - y$ puede no tener inverso y la uniformidad falla. Si $p \leq$ máx A, dos claves pueden ser iguales módulo p y colisionar siempre. Por eso se exige p primo y $p >$ máx A.

Aleatoriedad. Con a, b fijos, un adversario que conoce la función puede elegir claves que colisionen. Si a, b se eligen al azar después de fijar la entrada (o se mantienen secretos), para todo par de claves distintas la probabilidad de colisión es a lo más $1/m$, y la búsqueda esperada es $O(1 + \alpha)$.

4. (D) Comparación

- **División + encadenamiento:** mejor cuando $n > m$, hay muchas colisiones o eliminaciones frecuentes; peor si la memoria es ajustada o se necesita localidad de caché.
- **División + sondeo lineal:** mejor con α bajo, memoria ajustada y pocas eliminaciones; peor con α alto, muchas eliminaciones o entrada adversaria.
- **Universal + encadenamiento:** mejor cuando la entrada puede ser adversaria y se necesita una garantía probabilística; peor si no se pueden elegir a, b al azar o mantener secretos.

5. (a) Cálculo de la función prefijo (LPS) de $P =$ `ababaca`

Sea $P[1..7] = \langle a, b, a, b, a, c, a \rangle$. Inicialmente $k = 0$ y $\pi[1] = 0$.

q	comparaciones $P[k+1]$ vs $P[q]$	k final	$\pi[q]$
2	$P[1]{=}a \neq P[2]{=}b$	0	0
3	$P[1]{=}a = P[3]{=}a \Rightarrow k \leftarrow 1$	1	1
4	$P[2]{=}b = P[4]{=}b \Rightarrow k \leftarrow 2$	2	2
5	$P[3]{=}a = P[5]{=}a \Rightarrow k \leftarrow 3$	3	3
6	$P[4]{=}b \neq P[6]{=}c$; $k \leftarrow \pi[3]{=}1$; $P[2]{=}b \neq c$; $k \leftarrow \pi[1]{=}0$; $P[1]{=}a \neq c$	0	0
7	$P[1]{=}a = P[7]{=}a \Rightarrow k \leftarrow 1$	1	1

Por lo tanto:

$$\boxed{\pi \;=\; \langle\, 0,\; 0,\; 1,\; 2,\; 3,\; 0,\; 1 \,\rangle}.$$

Lectura: por ejemplo, $\pi[5] = 3$ indica que el mayor prefijo propio de `ababa` que también es sufijo es `aba` (largo 3).

6. (b) Ejecución de KMP-Matcher sobre $T =$ `abababacaba`

$n = 11$, $m = 7$, $\pi = \langle 0, 0, 1, 2, 3, 0, 1 \rangle$. Se recorre el texto de izquierda a derecha manteniendo q = largo del prefijo de P que coincide con el sufijo de $T[1..i]$:

i	$T[i]$	acción (mismatch $\Rightarrow q = \pi[q]$)	q final
1	a	$P[1]{=}a = T[1] \Rightarrow q \leftarrow 1$	1
2	b	$P[2]{=}b = T[2] \Rightarrow q \leftarrow 2$	2
3	a	$P[3]{=}a = T[3] \Rightarrow q \leftarrow 3$	3
4	b	$P[4]{=}b = T[4] \Rightarrow q \leftarrow 4$	4
5	a	$P[5]{=}a = T[5] \Rightarrow q \leftarrow 5$	5
6	b	$P[6]{=}c \neq b$; $q \leftarrow \pi[5]{=}3$; $P[4]{=}b = T[6] \Rightarrow q \leftarrow 4$	4
7	a	$P[5]{=}a = T[7] \Rightarrow q \leftarrow 5$	5
8	c	$P[6]{=}c = T[8] \Rightarrow q \leftarrow 6$	6
9	a	$P[7]{=}a = T[9] \Rightarrow q \leftarrow 7 = m$	**match**, $q \leftarrow \pi[7]{=}1$
10	b	$P[2]{=}b = T[10] \Rightarrow q \leftarrow 2$	2
11	a	$P[3]{=}a = T[11] \Rightarrow q \leftarrow 3$	3

En $i = 9$ se alcanza $q = m = 7$:

$$\text{shift} = i - m = 9 - 7 = 2.$$

Efectivamente, $T[3.,9] = \texttt{ababaca} = P$. No hay más ocurrencias.

$$\boxed{\text{Única ocurrencia: shift 2, es decir } T[3.,9]}\ .$$

7. (c) Complejidad de CLRS

Compute-Prefix-Function. El contador k crece a lo más una unidad por iteración del `for` (hasta m), y el `while` solo lo decrece vía $k = \pi[k]$. En total el `while` se ejecuta a lo más m veces (amortizado), luego el costo es

$$\Theta(m).$$

KMP-Matcher. Por el mismo argumento aplicado a q (crece a lo más una vez por posición de T y decae solo vía π), el bucle `for` de n pasos más los `while` acumulan

$$\Theta(n+m).$$

Conclusión. La función prefijo π (LPS) se calcula en $\Theta(m)$ y el matching completo corre en $\boxed{\Theta(n+m)}$, que es lineal en la longitud total del texto y el patrón.

8. (a) Depuración del rastreo de Compute-Prefix-Function

La tabla del estudiante es (valores incorrectos marcados):

q	k al inicio	evolución de k	$\pi[q]$ reportado
2	0	$P[1]{=}a \neq P[2]{=}b$	0
3	0	$P[1]{=}a = P[3]{=}a \Rightarrow k = 1$	1
4	1	$P[2]{=}b = P[4]{=}b \Rightarrow k = 2$	$\boxed{1}$ (debe ser 2)
5	1	$P[2]{=}b \neq P[5]{=}a;\ k = \pi[1] = 0;\ P[1]{=}a = P[5]{=}a \Rightarrow k = 1$	$\boxed{1}$ (debe ser 3)
6	1	$P[2]{=}b \neq P[6]{=}c;\ k = \pi[1] = 0;\ P[1]{=}a \neq P[6]{=}c$	0
7	0	$P[1]{=}a = P[7]{=}a \Rightarrow k = 1$	1

Los dos valores incorrectos son $\pi[4]$ y $\pi[5]$.

Causa raíz. En $q = 4$, el paso correcto es $k \leftarrow k + 1$ (línea `if` del pseudocódigo): como $P[k{+}1] = P[2]{=}b$ coincide con $P[4]{=}b$, debe quedar $k = 2$ y por tanto $\pi[4] = 2$. El estudiante anotó $k = 1$, propagando ese valor erróneo a $q = 5$: al entrar con $k = 1$, el algoritmo nunca compara $P[3]$ contra $P[5]$, de modo que $\pi[5]$ se reporta 1 en vez de 3. Las filas $q = 6$ y $q = 7$ *se recuperan por coincidencia* y quedan correctas.

Función prefijo correcta:

$$\boxed{\pi = \langle\, 0,\ 0,\ 1,\ 2,\ 3,\ 0,\ 1 \,\rangle}.$$

9. (b) Depuración del rastreo de KMP-Matcher

La tabla del estudiante es (valores incorrectos marcados):

i	$T[i]$	acción	q final reportado
1	a	$P[1]{=}a = T[1] \Rightarrow q = 1$	1
2	b	$P[2]{=}b = T[2] \Rightarrow q = 2$	2
3	a	$P[3]{=}a = T[3] \Rightarrow q = 3$	3
4	b	$P[4]{=}b = T[4] \Rightarrow q = 4$	4
5	a	$P[5]{=}a = T[5] \Rightarrow q = 5$	5
6	b	$P[6]{=}c \neq T[6];\ q = \pi[5] = 3;\ P[4]{=}b = T[6] \Rightarrow q = 4$	$\boxed{3}$ (debe ser 4)
7	a	$P[5]{=}a = T[7] \Rightarrow q = 5$	5
8	c	$P[6]{=}c = T[8] \Rightarrow q = 6$	6
9	a	$P[7]{=}a = T[9] \Rightarrow q = 7 = m$; imprime shift 2; $q = \pi[7] = 1$	$\boxed{7}$ (debe ser 1)
10	b	$P[2]{=}b = T[10] \Rightarrow q = 2$	2
11	a	$P[3]{=}a = T[11] \Rightarrow q = 3$	3

Los dos valores incorrectos están en $i = 6$ y $i = 9$.

- $i = 6$**:** el estudiante aplicó el retroceso $q = \pi[5] = 3$ pero *omitió el* `if` *posterior* (línea `If` de KMP-Matcher): como $P[q{+}1] = P[4]{=}b = T[6]{=}b$, debe quedar $q = 4$, no 3.
- $i = 9$**:** el estudiante detectó el match pero *omitió el reinicio* $q \leftarrow \pi[q]$ (línea después del `print`): tras reportar la ocurrencia en el shift 2, debe ser $q = \pi[7] = 1$, no $q = 7$.

Secuencia correcta de q:

$$\boxed{q = \langle\, 1,\ 2,\ 3,\ 4,\ 5,\ 4,\ 5,\ 6,\ 1,\ 2,\ 3 \,\rangle}.$$

La única ocurrencia se encuentra en $i = 9$, con

$$\boxed{\text{shift} = i - m = 9 - 7 = 2}.$$

10. (c) Por qué es peligroso no reiniciar q en $i = 9$

Si al imprimir el match se dejara $q = 7$ sin reiniciar, en $i = 10$ el algoritmo intentaría comparar $P[q{+}1] = P[8]$, que *no existe* (el patrón tiene largo $m = 7$): se accede fuera de rango del arreglo. Aun si la implementación lo evitara, $q = 7$ le haría creer que un prefijo completo del patrón sigue coincidiendo y reportaría desplazamientos espurios (falsos positivos) o fallaría el invariante $q \leq m$ que garantiza el costo lineal. Por eso, tras cada match, KMP restablece $q = \pi[q]$.

11. (a) Función prefijo de $S = P\#T$

Con $P =$ `ababaca` ($m = 7$) y $T =$ `abababacaba` ($n = 11$):

$$S = \texttt{a b a b a c a \# a b a b a b a c a b a}, \qquad |S| = m + 1 + n = 19.$$

Aplicando COMPUTE-PREFIX-FUNCTION al string S completo (recordar que en la tabla $S[k+1]$ cumple el rol de $P[k+1]$):

q	$S[q]$	evolución de k	$\pi[q]$
1	a	(caso base)	0
2	b	$S[1]{=}a \neq b$	0
3	a	$S[1]{=}a = a \Rightarrow k = 1$	1
4	b	$S[2]{=}b = b \Rightarrow k = 2$	2
5	a	$S[3]{=}a = a \Rightarrow k = 3$	3
6	c	$S[4]{=}b \neq c;\ k = \pi[3]{=}1;\ S[2]{=}b \neq c;\ k = \pi[1]{=}0;\ S[1]{=}a \neq c$	0
7	a	$S[1]{=}a = a \Rightarrow k = 1$	1
8	$\#$	$S[2]{=}b \neq \#;\ k = \pi[1]{=}0;\ S[1]{=}a \neq \#$	0
9	a	$S[1]{=}a = a \Rightarrow k = 1$	1
10	b	$S[2]{=}b = b \Rightarrow k = 2$	2
11	a	$S[3]{=}a = a \Rightarrow k = 3$	3
12	b	$S[4]{=}b = b \Rightarrow k = 4$	4
13	a	$S[5]{=}a = a \Rightarrow k = 5$	5
14	b	$S[6]{=}c \neq b;\ k = \pi[5]{=}3;\ S[4]{=}b = b \Rightarrow k = 4$	4
15	a	$S[5]{=}a = a \Rightarrow k = 5$	5
16	c	$S[6]{=}c = c \Rightarrow k = 6$	6
17	a	$S[7]{=}a = a \Rightarrow k = 7$	**7**
18	b	$S[8]{=}\# \neq b;\ k = \pi[7]{=}1;\ S[2]{=}b = b \Rightarrow k = 2$	2
19	a	$S[3]{=}a = a \Rightarrow k = 3$	3

$$\boxed{\pi(S) \;=\; \langle\, 0,0,1,2,3,0,1,\ 0,1,2,3,4,5,4,5,6,7,\ 2,3 \,\rangle}.$$

Observación: las primeras $m = 7$ entradas coinciden exactamente con el LPS de P calculado en el Enunciado 2.

12. (b) Ocurrencias detectadas

La condición de ocurrencia es $\pi[q] = m = 7$ en la zona del texto ($q \geq m + 2 = 9$). El único tal valor es

$$q = 17, \qquad \pi[17] = 7.$$

Conversión a índices de T. El carácter $T[i]$ ocupa la posición $m+1+i$ de S. Si la ocurrencia de largo m *termina* en q, entonces $T[i]$ con $i = q-(m+1)$ es el último carácter de la ocurrencia en el texto; su inicio (1-indexado) es

$$i - m + 1 = q - 2m, \qquad \text{shift} = \text{inicio} - 1 = q - 2m - 1.$$

Para $q = 17$: inicio en $T = 17 - 14 = 3$, shift $= 2$:

$$\boxed{T[3..9] = \texttt{ababaca} = P, \qquad \text{shift } 2}.$$

Esto *coincide* con el resultado del Enunciado 2 (única ocurrencia en el shift 2). No hay más posiciones con $\pi[q] = 7$ (los $\pi[18] = 2$ y $\pi[19] = 3$ corresponden a los prefijos `ab` y `aba`, incompletos).

13. (c) Por qué el # y el costo

Rol del centinela. El # es un carácter que no aparece en P. Si se computara π sobre PT sin el #, un prefijo de P podría casarse con un *sufijo que cruza la frontera* entre P y T, produciendo $\pi[q]$ mayores a los reales y *falsas ocurrencias*. Por ejemplo, sin #, al procesar el inicio de T se podría heredar el k acumulado de P y reportar matches que no existen. El # corta esa conexión: como # no aparece en T, ningún sufijo de la zona del texto puede extender un prefijo más allá de la posición $m + 1$, lo que garantiza

$$\pi[q] \leq m \qquad \text{para todo } q \text{ en la zona del texto,}$$

y $\pi[q] = m$ indica exactamente una ocurrencia que termina en q.

Costo. Se computa π de un string de largo $n + m + 1$ con el mismo argumento amortizado de siempre (el contador k crece a lo más una unidad por paso y decrece solo vía π):

Tiempo $\Theta(n + m)$, Espacio adicional $\Theta(n + m)$

para guardar π. Es el mismo costo asintótico de KMP, con la ventaja de usar una única pasada sobre $S = P\#T$.